

Recitation 8 BLE

1. BLE API introduction (Please refer to BLE_API.pdf in the recitations folder in Brightspace)
2. Library Resources: [Github Repo](#) for BLE
3. <https://os.mbed.com/docs/mbed-os/v5.15/apis/bluetooth.html> **Note: Please refer to the documentation and library github repo library files.**
4. Example problems <https://github.com/ARMmbed/mbed-os-example-ble.git>
5. **Code 1:** Initializes the BLE onboard and immediately starts broadcasting the data; doesn't care whether it's connected to devices or not.
6. **Code 2:** Initializes BLE on board and waits for the devices to connect, and then advertises the data
7. **Code 3:** Initializes the BLE on board and starts other work, such as recognising frequencies.
Whenever the device connects to the board, it advertises any up-to-date data present.

[GitHub Repo](#)

[BLE API Documentation](#)

EXAMPLES

Note: Please refer to the documentation and library GitHub repo library files.

Please download the Lightblue APP

For IOS: [App Store](#)

For Android: [Play Store](#)

Code 1:

```
#include "mbed.h"
#include "ble/BLE.h" // gives us Bluetooth Low Energy functionality
#include "ble/gatt/GattService.h" // Services are how BLE organises data
#include "ble/gatt/GattCharacteristic.h" //Characteristics hold the actual
data
#include "ble/Gap.h" //Handles how devices connect to each other
#include "ble/gap/AdvertisingDataBuilder.h" //helps create the "I'm here!"
broadcast message
#include "events/EventQueue.h" //helps schedule tasks and timing
#include <string.h>

BufferedSerial serial_port(USBTX, USBRX, 115200);
FileHandle *mbed::mbed_override_console(int) {
    return &serial_port;
}

// Make BLE and events functions available without prefixes
using namespace ble;
using namespace events;
```

```

// Get the single BLE instance that all code will use
BLE &ble_interface = BLE::Instance();
// Create our event queue for scheduling tasks
EventQueue event_queue;

DigitalOut led(LED1);

// Create unique IDs for our service and characteristic
// These long numbers ensure our service doesn't conflict with others
const UUID TREMOR_SERVICE_UUID("A0E1B2C3-D4E5-F6A7-B8C9-D0E1F2A3B4C5");
const UUID TREMOR_TYPE_CHAR_UUID("A1E2B3C4-D5E6-F7A8-B9C0-D1E2F3A4B5C6");

// Define our TREMOR _ strings
const char* NONE = "NONE";
const char* TREMOR_STRING = "TREMOR";
const char* DYSKINESIA_STRING = "DYSKINESIA";

// Maximum string length for our TREMOR type (including null terminator)
#define MAX_TREMOR_STRING_LEN 11

// Buffer to hold our TREMOR type string
// Initialize with "TREMOR"
uint8_t TREMORValue[MAX_TREMOR_STRING_LEN];

// Create our BLE characteristic for the TREMOR type string
// ReadOnly: Other devices can only read this value, not write to it
// Notify: Connected devices can be notified when this value changes
ReadOnlyArrayGattCharacteristic<uint8_t, MAX_TREMOR_STRING_LEN>
TREMORTypeCharacteristic(
    TREMOR_TYPE_CHAR_UUID,
    TREMORValue,
    GattCharacteristic::BLE_GATT_CHAR_PROPERTIES_NOTIFY
);

// Put our characteristic in a table
GattCharacteristic *charTable[] = { &TREMORTypeCharacteristic };

// Create our BLE service and add the characteristic to it
GattService TREMOR_Service(TREMOR_SERVICE_UUID, charTable, 1);

// Track whether we're in tremor (true) or dyskinesia (false) state
bool isTremor = true;

// Initialize with the tremor string
void init_TREMOR_value() {
    // Copy the tremor string to our buffer
    strcpy((char*)TREMORValue, NONE);
}

// This function sends a notification about the current TREMOR _ type
// It's called every second to toggle between tremor and dyskinesia
void send_TREMOR_notification() {
    // Set the string value based on our current state
    if (isTremor) {

```

```
        strcpy((char*)TREMORValue, TREMOR_STRING);
    } else {
        strcpy((char*)TREMORValue, DYSKINESIA_STRING);
    }

    // Send the notification to any connected devices
    ble_interface.gattServer().write(
        TREMORTypeCharacteristic.getValueHandle(),
        TREMORValue,
        strlen((char*)TREMORValue) + 1 // +1 for null terminator
    );

    // Print the current state for debugging
    printf("Sent notification: %s\n", (char*)TREMORValue);

    // Toggle the LED to show the state changed
    led = !led;

    // Switch between tremor and dyskinesia states
    isTremor = !isTremor;
}

// This function is called automatically when BLE finishes initializing
void on_ble_init_complete(BLE::InitializationCompleteCallbackContext
*params) {
    // Check if there was an error during initialization
    if (params->error != BLE_ERROR_NONE) {
        printf("BLE initialization failed.\n");
        return;
    }

    // Initialize our TREMOR value to "TREMOR"
    init_TREMOR_value();

    // Add our TREMOR _ service to the BLE server
    ble_interface.gattServer().addService(TREMOR_Service);

    // Set up the advertising (how our device announces itself)
    // First create a buffer to hold the advertising data
    uint8_t adv_buffer[LEGACY_ADVERTISING_MAX_SIZE];
    AdvertisingDataBuilder adv_data(adv_buffer);

    // Set standard BLE flags
    adv_data.setFlags();

    // Set the device name that will appear when scanning
    adv_data.setName("TREMOR-Monitor");

    // Configure how often we advertise (160 * 0.625ms = 100ms)
    // 0.625ms is the standard BLE time unit
    ble_interface.gap().setAdvertisingParameters(
        LEGACY_ADVERTISING_HANDLE,
        AdvertisingParameters(advertising_type_t::CONNECTABLE_UNDIRECTED,
adv_interval_t(160))
```

```

    );

    // Load our advertising data and start broadcasting
    ble_interface.gap().setAdvertisingPayload(LEGACY_ADVERTISING_HANDLE,
adv_data.getAdvertisingData());
    ble_interface.gap().startAdvertising(LEGACY_ADVERTISING_HANDLE);

    printf("BLE advertising started as TREMOR_-_Monitor\n");

    // Schedule our notification function to run every 1000ms (1 second)
    event_queue.call_every(1000ms, send_TREMOR_notification);
}

// This function handles BLE events by putting them in our event queue
/*Acts as a callback that the BLE system invokes when it has events that
need processing
(like connection requests, disconnections, data transmissions)
Takes a parameter context which contains information about the pending BLE
events
Rather than processing BLE events immediately, it puts them into the event
queue using event_queue.call()
The callback(&ble_interface, &BLE::processEvents) creates a function
pointer to the processEvents method
of the BLE interface*/
void schedule_ble_events(BLE::OnEventsToProcessCallbackContext *context) {
    event_queue.call(callback(&ble_interface, &BLE::processEvents));
}

int main() {

    printf("Starting BLE TREMOR _ Monitor...\n");

    // Set up our event handler for BLE events
    ble_interface.onEventsToProcess(schedule_ble_events);

    // Initialize BLE with our callback function
    ble_interface.init(on_ble_init_complete);

    // Start the event queue running forever
    // This keeps our program running and processing events
    event_queue.dispatch_forever();
}

/* Use the LightBlue app and search for the Bluetooth service name.
In the example code I provided, the Bluetooth name is "TREMOR-Monitor".
Subscribe to the advertisement data and convert the data from HEX to a
string.
You will then see the notifications coming from the board to the mobile
app via Bluetooth.
*/

```

Code 2:

```
#include "mbed.h"
#include "ble/BLE.h"
#include "ble/gatt/GattService.h"
#include "ble/gatt/GattCharacteristic.h"
#include "ble/Gap.h"
#include "ble/gap/AdvertisingDataBuilder.h"
#include "events/EventQueue.h"
#include <chrono>
#include <string.h> // For string functions

BufferedSerial serial_port(USBTX, USBRX, 115200);
FileHandle *mbed::mbed_override_console(int) {
    return &serial_port;
}

using namespace ble;
using namespace events;
using namespace std::chrono;

BLE &ble_interface = BLE::Instance();
EventQueue event_queue;
DigitalOut led(LED1);

const UUID TREMOR_SERVICE_UUID("A0E1B2C3-D4E5-F6A7-B8C9-D0E1F2A3B4C5");
const UUID TREMOR_TYPE_CHAR_UUID("A1E2B3C4-D5E6-F7A8-B9C0-D1E2F3A4B5C6");

// Define our TREMOR _ strings
const char* TREMOR_STRING = "TREMOR";
const char* DYSKINESIA_STRING = "DYSKINESIA";

// Maximum string length for our TREMOR type (including null terminator)
#define MAX_TREMOR_STRING_LEN 11

// Buffer to hold our TREMOR type string
uint8_t TREMORValue[MAX_TREMOR_STRING_LEN];

// Update the characteristic to use our string buffer with appropriate
size
ReadOnlyArrayGattCharacteristic<uint8_t, MAX_TREMOR_STRING_LEN>
TREMORTypeCharacteristic(
    TREMOR_TYPE_CHAR_UUID,
    TREMORValue,
    GattCharacteristic::BLE_GATT_CHAR_PROPERTIES_NOTIFY
);

GattCharacteristic *charTable[] = { &TREMORTypeCharacteristic };
GattService TREMOR_Service(TREMOR_SERVICE_UUID, charTable, 1);

bool isTremor = true;
Ticker notification_ticker;
bool device_connected = false;
```

```

void send_TREMOR_notification() {
    if (!device_connected) {
        printf("No device connected, skipping notification\n");
        return;
    }

    // Set the string value based on our current state
    if (isTremor) {
        strcpy((char*)TREMORValue, TREMOR_STRING);
    } else {
        strcpy((char*)TREMORValue, DYSKINESIA_STRING);
    }

    ble_interface.gattServer().write(
        TREMORTypeCharacteristic.getValueHandle(),
        TREMORValue,
        strlen((char*)TREMORValue) + 1 // +1 for null terminator
    );

    printf("Sent notification: %s\n", (char*)TREMORValue);

    led = !led;

    isTremor = !isTremor;
}

//ConnectionEventHandler class is based on the ble::Gap::EventHandler
class from the BLE library.

// Class to handle BLE connection events by extending Gap::EventHandler
// Provides custom behavior for when devices connect and disconnect
class ConnectionEventHandler : public ble::Gap::EventHandler {
public:
    // Called automatically by BLE stack when a device connects
    // "virtual" means we're overriding the parent class method
    virtual void onConnectionComplete(const
ble::ConnectionCompleteEvent &event) {
        // Check if connection was successful (no errors)
        if (event.getStatus() == BLE_ERROR_NONE) {
            printf("Device connected!\n");

            // Update flag so our app knows a device is connected
            device_connected = true;

            // Initialize with tremor string
            strcpy((char*)TREMORValue, TREMOR_STRING);

            // Set up a timer to send notifications every 1 second
            // notification_ticker is an instance of the Ticker class
            from the mbed library
            notification_ticker.attach([]() {
                // Add notification function to event queue instead of
                calling directly
                // This is safer than executing from interrupt context

```

```

        event_queue.call(send_TREMOR_notification);
    }, 1s); // 1s is a chrono literal for one second
}

// Called automatically when a device disconnects
virtual void onDisconnectionComplete(const
ble::DisconnectionCompleteEvent &event) {
    printf("Device disconnected!\n");

    // Update flag so notifications stop
    device_connected = false;

    // Stop the notification timer
    notification_ticker.detach();

    // Start advertising again so new devices can connect
    ble_interface.gap().startAdvertising(ble::LEGACY_ADVERTISING_HANDLE);
    printf("Restarted advertising\n");
}

};

// Create an instance of our handler class
ConnectionEventHandler connection_handler;

// This function gets called when BLE initialization completes
void on_ble_init_complete(BLE::InitializationCompleteCallbackContext
*params) {
    // Check if there was an initialization error
    if (params->error != BLE_ERROR_NONE) {
        printf("BLE initialization failed.\n");
        return;
    }

    // Initialize our string buffer with TREMOR
    strcpy((char*)TREMORValue, TREMOR_STRING);

    // Register our service with the BLE GATT server
    ble_interface.gattServer().addService(TREMOR_Service);

    // Create buffer for advertising data - size is defined by BLE
spec uint8_t adv_buffer[LEGACY_ADVERTISING_MAX_SIZE];

    // Helper object to build properly formatted advertising data
    AdvertisingDataBuilder adv_data(adv_buffer);

    // Set standard BLE flags required in advertising packet
    adv_data.setFlags();

    // Set device name that shows up when scanning
    adv_data.setName("TREMOR--Monitor");

```

```
// Configure advertising parameters:
// - LEGACY_ADVERTISING_HANDLE: Standard advertising type
// - CONNECTABLE_UNDIRECTED: Any device can connect to us
// - adv_interval_t(160): Advertise every 100ms (160 × 0.625ms)
ble_interface.gap().setAdvertisingParameters(
    LEGACY_ADVERTISING_HANDLE,

AdvertisingParameters(advertising_type_t::CONNECTABLE_UNDIRECTED,
adv_interval_t(160))
);

// Set the actual advertising packet content
ble_interface.gap().setAdvertisingPayload(
    LEGACY_ADVERTISING_HANDLE,
    adv_data.getAdvertisingData()
);

// Register our connection handler to process connection events
ble_interface.gap().setEventHandler(&connection_handler);

// Start broadcasting our presence
ble_interface.gap().startAdvertising(LEGACY_ADVERTISING_HANDLE);

printf("BLE advertising started as TREMOR_-_Monitor");
printf("Waiting for device connection...\n");
}

// This function handles BLE system events
void schedule_ble_events(BLE::OnEventsToProcessCallbackContext
*context) {
    // Add BLE events to our event queue instead of processing
    immediately
    // This ensures events are handled in the main thread, not
    interrupt context
    event_queue.call(callback(&ble_interface, &BLE::processEvents));
}

int main() {

    printf("Starting BLE TREMOR _ Monitor...\n");

    // Set up our event handler for BLE events
    ble_interface.onEventsToProcess(schedule_ble_events);

    // Initialize BLE with our callback function
    ble_interface.init(on_ble_init_complete);

    // Start the event queue running forever
    // This keeps our program running and processing events
    event_queue.dispatch_forever();
}
```


Code 3:

```
#include "mbed.h"
#include "ble/BLE.h"
#include "ble/gatt/GattService.h"
#include "ble/gatt/GattCharacteristic.h"
#include "ble/Gap.h"
#include "ble/gap/AdvertisingDataBuilder.h"
#include "events/EventQueue.h"
#include <chrono>
#include <string>

BufferedSerial serial_port(USBTX, USBRX, 115200);

FileHandle *mbed::mbed_override_console(int) {
    return &serial_port;
}

// Bring in namespaces so we don't have to type them each time
using namespace ble;
using namespace events;
using namespace std::chrono;

// Get the BLE instance and set up event queue
BLE &ble_interface = BLE::Instance();
EventQueue event_queue;
// LED for visual feedback when counter updates
DigitalOut led(LED1);

// Unique IDs for our service and characteristic – these are made-up
// values
const UUID TREMOR_SERVICE_UUID("A0E1B2C3-D4E5-F6A7-B8C9-D0E1F2A3B4C5");
const UUID STATUS_MSG_CHAR_UUID("A1E2B3C4-D5E6-F7A8-B9C0-D1E2F3A4B5C6");

// Main counter and stats counters
uint8_t counter = 0;
uint8_t preMedCount = 0;
uint8_t Dyskinesia = 0;

// Buffer to hold our status message – big enough for a detailed message
uint8_t statusBuffer[100] = {0};
uint8_t statusLength = 0;

// BLE characteristic – allows read and notify operations
ReadOnlyArrayGattCharacteristic<uint8_t, sizeof(statusBuffer)>
statusMsgCharacteristic(
    STATUS_MSG_CHAR_UUID,
    statusBuffer,
    GattCharacteristic::BLE_GATT_CHAR_PROPERTIES_NOTIFY
);

// Add characteristic to table and create service
```

```
GattCharacteristic *charTable[] = { &statusMsgCharacteristic };
GattService tremorService(TREMOR_SERVICE_UUID, charTable, 1);

// Timer for regular counter updates
Ticker counter_ticker;
// Flag to track if anyone's connected
bool device_connected = false;

// Creates and sends the status message with current state
void update_status_message() {
    // Determine state based on counter value
    // Using modulo to create patterns - every 3rd count is pre-med, every
    5th is dyskinesia
    bool isPre = (counter % 3 == 0);
    bool isPost = (counter % 5 == 0);
    char stateStr[20] = "";

    // Handle all possible state combinations
    if (isPre && isPost) {
        strcpy(stateStr, "Pre+Dyskinesia");
    } else if (isPre) {
        strcpy(stateStr, "Pre-med");
    } else if (isPost) {
        strcpy(stateStr, "Dyskinesia");
    } else {
        strcpy(stateStr, "Neither");
    }

    // Format all the data into our status buffer
    snprintf((char*)statusBuffer, sizeof(statusBuffer),
             "Count: %d, State: %s, Pre: %d, dys: %d",
             counter, stateStr, preMedCount, Dyskinesia);
    statusLength = strlen((char*)statusBuffer);

    // Print locally for debugging
    printf("%s\n", statusBuffer);

    // Only send notification if a device is connected
    if (device_connected) {
        ble_interface.gattServer().write(
            statusMsgCharacteristic.getValueHandle(),
            statusBuffer,
            statusLength
        );
        printf("Notification sent\n");
    }
}

// Called every second to update the counter and state
void update_counter() {
    counter++;
    // Reset after 100 to keep values in reasonable range
    if (counter > 100) {
        counter = 1;
    }
}
```

```
        preMedCount = 0;
        Dyskinesia = 0;
    }

    // Check states and update corresponding counters
    bool isPre = (counter % 3 == 0);
    bool isPost = (counter % 5 == 0);

    if (isPre) preMedCount++;
    if (isPost) Dyskinesia++;

    // Toggle LED for visual feedback
    led = !led;

    // Update and possibly send the status message
    update_status_message();
}

// Handler for connection events
class ConnectionEventHandler : public ble::Gap::EventHandler {
public:
    // Called when a device connects
    virtual void onConnectionComplete(const ble::ConnectionCompleteEvent
&event) {
        if (event.getStatus() == BLE_ERROR_NONE) {
            printf("Device connected!\n");

            device_connected = true;

            // Send initial status right away so client has data
            update_status_message();
        }
    }

    // Called when a device disconnects
    virtual void onDisconnectionComplete(const
ble::DisconnectionCompleteEvent &event) {
        printf("Device disconnected!\n");

        device_connected = false;

        // Restart advertising so others can find us

ble_interface.gap().startAdvertising(ble::LEGACY_ADVERTISING_HANDLE);
        printf("Restarted advertising\n");
    }
};

// Create our connection handler
ConnectionEventHandler connection_handler;

// Called when BLE finishes initializing
void on_ble_init_complete(BLE::InitializationCompleteCallbackContext
*params) {
```

```
if (params->error != BLE_ERROR_NONE) {
    printf("BLE initialization failed.\n");
    return;
}

// Register our service with BLE
ble_interface.gattServer().addService(tremorService);

// Create advertising data buffer
uint8_t adv_buffer[LEGACY_ADVERTISING_MAX_SIZE];

// Helper to build the advertising data
AdvertisingDataBuilder adv_data(adv_buffer);

// Set required flags
adv_data.setFlags();

// Name that shows up in scanning
adv_data.setName("Tremor-Counter");

// Set advertising parameters - 100ms intervals
ble_interface.gap().setAdvertisingParameters(
    LEGACY_ADVERTISING_HANDLE,
    AdvertisingParameters(advertising_type_t::CONNECTABLE_UNDIRECTED,
adv_interval_t(100))
);

// Set the payload (the actual data)
ble_interface.gap().setAdvertisingPayload(
    LEGACY_ADVERTISING_HANDLE,
    adv_data.getAdvertisingData()
);

// Register our connection handler
ble_interface.gap().setEventHandler(&connection_handler);

// Start broadcasting our presence
ble_interface.gap().startAdvertising(LEGACY_ADVERTISING_HANDLE);

printf("BLE advertising started as Tremor-Counter\n");
printf("Starting counter from 1 to 100...\n");

// Reset counter and start the ticker
counter = 0;
counter_ticker.attach([]() {
    event_queue.call(update_counter);
}, 1s);
}

// Handle BLE events by adding them to our event queue
void schedule_ble_events(BLE::OnEventsToProcessCallbackContext *context) {
    event_queue.call(callback(&ble_interface, &BLE::processEvents));
}
```

```
// Program starts here
int main() {
    printf("Starting BLE Tremor Counter...\n");

    // Set up event handler for BLE
    ble_interface.onEventsToProcess(schedule_ble_events);

    // Initialize BLE with our callback
    ble_interface.init(on_ble_init_complete);

    // Run the event queue forever – this loops until power off
    event_queue.dispatch_forever();
}
```